

CloudPool

The Design

Written For Community

May 30, 2026

Contents

Introduction	4
1 What We're Actually Building	5
1.1 The Three Core Services We're Providing	5
1.1.1 File Storage (But Different)	5
1.1.2 Relational Database	6
1.1.3 Semantic Search	6
2 Why This Problem Matters	7
2.1 The Cost Problem	7
2.1.1 AWS Costs	7
2.1.2 CloudPool Costs	7
2.2 The Lock-In Problem	7
2.3 The Privacy Problem	8
3 The Architecture: Why We Chose What We Chose	9
3.1 Spring Boot for the API Layer	9
3.1.1 The Authentication Reality	9
3.2 PostgreSQL for Relational Data	10
3.3 Google Drive for File Storage	10
3.4 Weaviate for Vector Search	11
4 The Real Challenges: Edge Cases That Will Bite	12
4.1 Challenge 1: The Google Drive Pool Problem	12
4.1.1 Scenario 1: Contributor A Deletes Their Account	12
4.1.2 Scenario 2: Contributor A Runs Out of Space	12
4.1.3 Scenario 3: Google Drive API Rate Limits	12
4.1.4 Scenario 4: Google Drive API Changes or Goes Down	13
4.1.5 Scenario 5: Data Corruption	13
4.2 Challenge 2: Multi-Tenancy Isolation	13
4.2.1 Scenario 1: Accidental Tenant Leakage	13
4.2.2 Scenario 2: Shared Resources Across Tenants	13
4.2.3 Scenario 3: Quota Sharing	14
4.3 Challenge 3: Async Operations Going Wrong	14
4.3.1 Scenario 1: Async Job Fails	14
4.3.2 Scenario 2: Jobs Processing Out of Order	14
4.3.3 Scenario 3: Job State Becomes Inconsistent	14
4.4 Challenge 4: Search Queries Hitting Edge Cases	15
4.4.1 Scenario 1: A User Searches for a Very Rare Document	15
4.4.2 Scenario 2: Multilingual Search	15
4.4.3 Scenario 3: Domain-Specific Vocabulary	15
4.4.4 Scenario 4: Document Changes	15
4.4.5 Scenario 5: Embedding Model Updates	15
5 Database Design Rationale	16
5.1 Core Entities	16

5.1.1	Users	16
5.1.2	Buckets and Files	16
5.1.3	Dynamic Tables	17
5.1.4	Vector Collections	18
5.2	Multi-Tenancy at the Database Level	18
6	API Design: REST vs GraphQL	19
6.1	REST API: For Simple Cases	19
6.2	GraphQL API: For Complex Cases	19
6.3	Handling Complexity in Queries	20
7	Handling Real-World Failures	21
7.1	Retry Logic	21
7.2	Graceful Degradation	21
7.3	Circuit Breakers	21
8	Multi-Tenancy in Practice	23
8.1	The Tenant Context	23
8.2	Testing Tenant Isolation	23
9	Scaling: How We Handle Growth	25
9.1	Single Instance Phase (1-1,000 users)	25
9.2	Multi-Instance Phase (1,000-100,000 users)	25
9.3	Distributed Phase (100,000+ users)	25
10	Security: Layered Approach	27
10.1	Authentication: Verifying Identity	27
10.1.1	Password-based auth	27
10.1.2	Better: Multi-factor authentication	27
10.1.3	Tokens	27
10.2	Authorization: Verifying Permissions	27
10.2.1	Resource ownership	27
10.2.2	Permissions	28
10.2.3	Roles	28
10.3	Data Protection: Encryption	28
10.3.1	With encryption	28
10.3.2	Without encryption	28
10.4	Input Validation	28
10.4.1	Example: File upload	28
10.4.2	Example: Query parameter	28
11	Operational Reality: Running This Thing	30
11.1	Monitoring	30
11.2	Debugging	30
11.3	Maintenance Windows	30
12	Edge Cases Nobody Expects	32
12.1	Unicode in Filenames	32
12.2	Concurrent Writes to Same Record	32
12.3	Accidental Sharing of Public Links	32
12.4	Files Disappearing	32
13	The Path From MVP to Production	34

13.1 Phase 1: MVP	34
13.2 Phase 2: Make it Real	34
13.3 Phase 3: Add Features	34
13.4 Phase 4: Scale	34
13.5 Phase 5+: Harden	35
14 Cost Analysis: Is This Actually Free?	36
14.1 Storage	36
14.2 Compute	36
14.3 Database	36
14.4 Embeddings	36
14.5 Bandwidth	36
14.6 Real cost for self-hosted	36
14.7 Real cost for managed service	36
14.8 AWS equivalent cost	36
14.9 Savings	36

Introduction

Look, I'm going to be honest with you about what we're building here and why. CloudPool isn't some flashy new idea that's going to change the world overnight. It's a pragmatic answer to a very real problem that developers face every single day.

Let me start with the problem because that's what matters. You've probably experienced this yourself. You build an application. It starts small. You throw it on Firebase or AWS because it's easy. You don't have to think about infrastructure, and that's amazing when you're just getting started. But then your application actually becomes successful. And suddenly you're getting hit with bills that make your eyes water.

I've seen startups spend more on AWS bills than on actual product development. I've seen engineers forced to spend weeks optimizing queries just to shave \$200 off the monthly bill. And here's the worst part: by the time you realize how expensive it's gotten, you're completely locked in. Your entire architecture is built on their proprietary APIs. Your data is in their format. Switching would cost you months of engineering time and potentially hundreds of thousands of dollars.

So we asked ourselves: what if there was another way?

Chapter 1

What We're Actually Building

CloudPool is a Backend-as-a-Service platform, but not in the way you might think. We're not trying to compete with AWS or Firebase head-to-head. That would be insane. We don't have their resources, their scale, or their engineering teams.

Instead, we're doing something different. We're creating a platform that lets developers build real applications without getting destroyed by costs, without losing control of their data, and without being trapped by vendor lock-in.

Here's the core insight: most developers don't actually need AWS's massive infrastructure. They need:

1. A place to store files
2. A database for their application
3. A way to search through data
4. Authentication that works
5. Something that scales reasonably

That's it. Everything else is nice-to-have but not essential.

1.1 The Three Core Services We're Providing

1.1.1 File Storage (But Different)

Traditional file storage services charge you per gigabyte. It adds up fast. But what if we used an approach that already exists and is free? Google Drive gives every account 15GB for free. That's a fact. Microsoft offers something similar with OneDrive. Dropbox has free tiers.

What if we created a system where users could contribute their own free storage to a shared pool? So one user contributes 15GB, another contributes 15GB, another contributes 15GB, and suddenly you have a shared pool of storage that costs literally nothing.

This works because:

- Users aren't uploading their personal files. They're contributing to a community resource
- The files are distributed across multiple accounts, so there's built-in redundancy
- If one person leaves the pool, their replicas exist on other accounts
- We're not beholden to any cloud provider's pricing

Yes, there are complications. We'll get to those. But the core idea solves a fundamental problem: how do you store files without going broke?

1.1.2 Relational Database

The second piece is actually storing structured data. Almost every application needs this. You need to store user information, product data, relationships between entities, all that stuff.

PostgreSQL already does this perfectly. It's been around for 30 years. It's incredibly reliable. But it's a database server that needs to run somewhere. So we run it. You connect to our instance. We handle the backups, the updates, the maintenance. You just use it.

The trick with using a single PostgreSQL instance for multiple tenants is multitenancy. Every table has a `tenant_id` column. Every single query is automatically filtered by tenant. This is enforced at the database level using row-level security policies. So even if there's a bug in our application code, the database itself prevents one tenant from accessing another tenant's data.

1.1.3 Semantic Search

The third piece is semantic search. This is the fancy one. Basically, you can store documents, and then ask the system “find me documents similar to this”. Not keyword matching. Actual semantic similarity.

It works like this: you upload a document, we convert it to a vector using embeddings, we store that vector in a specialized database designed for vector search, and then when you search, we convert your search query to a vector and find the most similar documents.

This is useful for things like:

- Finding similar documents in a knowledge base
- Recommendation systems
- Duplicate detection
- Content moderation
- And many other things

Chapter 2

Why This Problem Matters

Let me give you some concrete numbers so you understand why this matters.

2.1 The Cost Problem

Let's say you're building a social media application. Not huge. Maybe 100,000 monthly active users. That's decent scale but not massive.

2.1.1 AWS Costs

- RDS for database: \$3,000/month (t3.large instance, multi-AZ)
- S3 for file storage: \$2,000/month (assuming 100GB of data with some bandwidth)
- CloudFront for CDN: \$500/month
- DynamoDB for caching: \$500/month
- Lambda for processing: \$300/month
- Data transfer: \$500/month
- **Total:** \$6,800/month = \$81,600/year

That's the bare minimum. Once you add redundancy, backups, monitoring, more features, you're easily at \$100k+/year.

2.1.2 CloudPool Costs

- 100 users contributing to the pool
- Each user has 15GB free Google Drive space
- Pool total: 1500GB
- Your 100GB of storage: Free (fits in the pool)
- Managed postgres instance: Some monthly fee for the service
- Embeddings (if you use search): Minimal cost for API calls
- **Total:** Essentially free to \$100-500/month maximum

That's a 40-100x cost difference. For a bootstrapped startup, that's the difference between sustainable and unsustainable.

2.2 The Lock-In Problem

Here's something that bothers me about AWS and Firebase. Once you're on them, switching is incredibly painful. Why?

- Your data is in their format

- Your APIs are proprietary
- Your infrastructure is tightly coupled
- You’ve probably lost institutional knowledge about how to run these things

So when they raise prices, you can’t really leave. You’re stuck. They know you’re stuck. And prices tend to go up, not down.

With CloudPool, your data is in standard PostgreSQL format. Your files are just files on Google Drive. Your code that talks to our API is simple REST or GraphQL. If you decided to leave tomorrow, you could. It might take a few weeks to migrate everything, but it’s actually possible.

2.3 The Privacy Problem

I’m not going to go on a rant about how cloud providers are reading your data or anything. But the fact is, your data is sitting in their data centers. In their jurisdiction. Subject to their terms of service. Subject to government requests.

Some organizations can’t accept that. Banks, healthcare companies, organizations in certain countries, they need their data to stay in their control. With CloudPool, you can run the entire thing on your own servers if you want. Or you can contribute your own storage to the pool. You have options.

Chapter 3

The Architecture: Why We Chose What We Chose

Let me walk you through our architecture and explain the reasoning behind each choice.

3.1 Spring Boot for the API Layer

We chose Spring Boot as our main application framework. Some people might say “why Java in 2024?” and that’s a fair question. But here’s the thing: Spring Boot is incredibly mature, incredibly stable, and has a huge community.

When you’re building something like CloudPool, you don’t want cutting-edge. You want battle-tested. You want to know that when someone files a bug report, there are ten thousand other developers who’ve hit that exact bug. You want excellent documentation. You want security updates to come out within hours of a vulnerability being discovered.

Spring Boot gives you all of that. Plus, most enterprise developers know Java. If you’re trying to build an open-source project that people will actually contribute to, you need to make it accessible to people. Java is accessible.

But here’s the thing: Spring Boot alone isn’t special. What matters is what we do with it. We’re using it to:

1. **Accept requests** — Whether GraphQL or REST
2. **Authenticate users** — Validate JWT tokens, check API keys
3. **Authorize actions** — Make sure users can only do what they’re allowed to do
4. **Coordinate services** — Route requests to the right backend services
5. **Manage multi-tenancy** — Make sure one user’s data never touches another user’s data

Each of these is critical. And each of these has complexities that aren’t immediately obvious.

3.1.1 The Authentication Reality

When I say “validate JWT tokens”, that sounds simple. And the happy path is simple. But real authentication is much more complicated.

Here’s what actually happens:

A user logs in. We generate a JWT token. That token is valid for 15 minutes. After 15 minutes, the token expires. So the user needs to refresh it. But what if they log out? Well, the refresh token gets invalidated, but the JWT token is still technically valid for another 10 minutes. During those 10 minutes, could they still use the old token? Yes. Is that acceptable? Maybe. Depends on your security requirements.

Or what if the user’s account gets compromised? And a hacker has their refresh token? Well, the hacker can generate new access tokens indefinitely. So we need a way to revoke refresh tokens immediately.

Or what if the user resets their password? Do all their tokens get invalidated? Probably should.

Or what if the user’s role changes (they get promoted to admin)? When does that take effect? Immediately? Or after the token expires?

These are the kinds of questions that don’t have simple answers. You need to think through your security model carefully.

3.2 PostgreSQL for Relational Data

PostgreSQL is boring technology. And that’s why it’s the right choice.

Boring means:

- It’s not going away
- The documentation is comprehensive
- There are no major surprises
- It works the same way in all environments
- Everyone knows SQL

We could have used NoSQL databases. They’re sexy. They scale in certain ways that SQL databases don’t. But they also:

- Don’t support ACID transactions
- Make certain queries much harder
- Don’t support relationships well
- Require you to think about data structure differently

For CloudPool, we want relational data. We want ACID compliance. We want users to be able to query their data in standard SQL. PostgreSQL is the right tool.

The trick with using a single PostgreSQL instance for multiple tenants is multitenancy. Every table has a `tenant_id` column. Every single query is automatically filtered by tenant. This is enforced at the database level using row-level security policies. So even if there’s a bug in our application code, the database itself prevents one tenant from accessing another tenant’s data.

3.3 Google Drive for File Storage

This is the weird one. Why would we use Google Drive for file storage?

The answer is simple: it’s free and it’s already there.

Google gives every account 15GB of free storage. That’s a lot. And it’s free. AWS S3 costs money. Blob storage costs money. But Google Drive is just... free.

Now, you might be thinking “but Google can delete your account” or “Google could change their terms” or “what if Google shuts down Google Drive?” These are all valid concerns. And we need to handle them.

First, regarding Google deleting accounts: Google is very protective of their terms of service. They don’t randomly delete accounts. Your account gets deleted if you violate their policies. So

we build CloudPool in a way that respects those policies. We don't store illegal content. We don't use Google Drive for anything inappropriate.

Second, regarding terms changes: this is a real risk. But here's the thing: we're designing CloudPool to be adaptable. If Google changes their terms and we can't use their free tier anymore, we can add support for other providers. Azure has free OneDrive storage. Dropbox has free tier storage. IPFS exists. We're not locked into Google. We're just starting there because it makes sense right now.

Third, regarding Google shutting down: they probably won't. But if they do, all the files are still on users' personal Google Drive accounts. Users can download them whenever they want. We're not locked into anything.

The benefit of using Google Drive is that we get infinite scalability for free. Want to add 10,000 users? Each contributes their free 15GB. Suddenly we have 150TB of storage. Cost to us: zero.

3.4 Weaviate for Vector Search

Vector databases are newer technology. We're using Weaviate because it's:

- Open source
- Self-hostable
- Mature enough to be reliable
- Good documentation
- Actively maintained

Vector search is useful but not essential for CloudPool's MVP. However, it's a feature that unlocks interesting use cases. So we include it.

The idea is simple: convert text to vectors (using OpenAI's embedding API or a local model), store those vectors in Weaviate, then when you search, convert the search query to a vector and find similar documents.

This is different from full-text search (which searches for keywords) or fuzzy search (which finds typos). This finds documents that are semantically similar to your query.

Chapter 4

The Real Challenges: Edge Cases That Will Bite

Now let's talk about the hard stuff. Building CloudPool is straightforward in the happy path. But the real complexity comes from all the ways things can go wrong.

4.1 Challenge 1: The Google Drive Pool Problem

Let's say you have 100 users who've each contributed 15GB to the pool. That's 1500GB total. One user uploads a 500GB file. The system needs to decide which Google Drive accounts to spread it across.

The naive approach: just use the first available contributors. Put 100GB on contributor A, 100GB on contributor B, 100GB on contributor C, 100GB on contributor D, 100GB on contributor E.

But what happens when:

4.1.1 Scenario 1: Contributor A Deletes Their Account

Now 100GB of the file is gone. Permanently. The user's file is corrupted. Their data is lost.

The fix: replicate the file. Store it in multiple places. So you store the file in 3 locations. If one location goes down, you have 2 backups.

But now you're using 3x the storage space for every file. That's inefficient.

Alternative fix: replicate only critical files. But how do you decide what's critical?

Better fix: replicate with a replication factor that the user can choose. You want high redundancy? You get 5 copies. You want to save space? You get 1 copy (but accept the risk).

4.1.2 Scenario 2: Contributor A Runs Out of Space

They were contributing 15GB but they upgraded their plan to 200GB and now they're using 190GB of that. They only have 10GB available. When they've contributed to 2 different 100GB files, they're back to zero available space.

The fix: monitor contributor capacity continuously. When they get below 10GB available, stop assigning new files to them. Move existing files off their account to other contributors.

But this requires background jobs constantly running. This requires coordination. This requires keeping track of which files are on which drives.

4.1.3 Scenario 3: Google Drive API Rate Limits

Google has rate limits on their API. You can't make unlimited requests. So when many users are uploading simultaneously, you hit the rate limit. Requests start failing.

The fix: implement a queue. When uploads come in, put them in a queue. Process them at a rate that respects Google’s limits.

But now uploads aren’t instant. They’re queued. Users see delays.

Better fix: implement exponential backoff. When you hit a rate limit, wait 2 seconds, then retry. If it fails again, wait 4 seconds, then retry. Keep doubling until it works.

4.1.4 Scenario 4: Google Drive API Changes or Goes Down

Google deprecates an API endpoint. Or Google has an outage.

The fix: handle failures gracefully. When Google Drive is unavailable, don’t fail the entire system. Fall back to other contributors if they’re available.

4.1.5 Scenario 5: Data Corruption

A file is uploaded successfully. But when downloaded, the checksum doesn’t match. The file is corrupted.

The fix: calculate checksums on upload and download. Compare them. If they don’t match, retry the download. If it still fails, try downloading from a replica.

But you need to know which replicas exist. You need to track that in the database.

4.2 Challenge 2: Multi-Tenancy Isolation

You have 1000 tenants on the same PostgreSQL instance. They’re all querying the same database. Every single query needs to be filtered by `tenant_id`.

4.2.1 Scenario 1: Accidental Tenant Leakage

You write a query like `SELECT * FROM files WHERE user_id = ?` without filtering by `tenant_id`. If user IDs are sequential, a hacker might guess other users’ IDs and access their files.

The fix: implement row-level security at the database level. Every table has a policy that automatically filters by the current tenant. This is enforced by PostgreSQL itself, not by your application.

But row-level security adds complexity. It adds slight performance overhead. You need to understand how it works.

4.2.2 Scenario 2: Shared Resources Across Tenants

Most resources should be isolated. But some resources might be shared. Like vector embeddings. If tenant A indexes a document “The quick brown fox”, and tenant B tries to index the exact same text, should you:

- Generate the embedding twice (wasteful)
- Reuse the embedding (potential privacy leakage if embeddings leak tenant info)
- Generate it once but track ownership separately (complex)

The answer depends on your threat model. For most applications, the performance benefit of reusing embeddings outweighs the privacy risk.

4.2.3 Scenario 3: Quota Sharing

If you have 100 tenants and 1500GB of total storage pool space, do you:

- Give each tenant 15GB and prevent them from using more (fairness)
- Let them use unlimited space until the pool fills up (flexibility)
- Charge for usage (requires billing system)
- Implement a reputation system where active contributors get more quota (complex)

Each approach has trade-offs. We recommend fairness (per-user quota) for the MVP.

4.3 Challenge 3: Async Operations Going Wrong

Files get uploaded. You want to:

- Generate a thumbnail
- Index them for search
- Compress them
- Update statistics

All of this should happen asynchronously so the user gets a response quickly.

4.3.1 Scenario 1: Async Job Fails

A job is queued to generate a thumbnail. The job starts. But then something goes wrong. The image library crashes. The job dies.

The user uploaded their file. But it never got a thumbnail. Should they see an error? Should the system retry?

Better approach: implement retries with exponential backoff. If a job fails, wait 5 seconds and retry. If it fails again, wait 10 seconds. Keep trying up to 3 times. If all retries fail, mark it as failed and alert someone.

4.3.2 Scenario 2: Jobs Processing Out of Order

A file gets uploaded and queued for processing. Then the user updates the file's metadata and queues that for processing. But due to the way the queue works, the metadata update processes first, before the file upload.

The result: the metadata is updated, but the file hasn't been uploaded yet.

The fix: use ordering guarantees in your queue. Ensure that for a given file, jobs are processed in order.

4.3.3 Scenario 3: Job State Becomes Inconsistent

A job to update statistics in Redis is halfway through. The system crashes. Now Redis has partial data. This inconsistency could cause problems later.

The fix: make jobs idempotent. A job should be safe to run multiple times and produce the same result. So if the system crashes and retries the job, it doesn't cause double-counting or other issues.

4.4 Challenge 4: Search Queries Hitting Edge Cases

4.4.1 Scenario 1: A User Searches for a Very Rare Document

The search returns nothing. But the user swears they uploaded it. Did the search index fail? Did the embedding fail? Did the document get corrupted?

The fix: implement search quality monitoring. Track which documents were indexed successfully. Track which searches had results vs. no results.

4.4.2 Scenario 2: Multilingual Search

A user uploads a document in Mandarin. Then searches for the same content in English. The search should ideally work, but it won't if your embeddings are English-only.

The fix: use multilingual embeddings (like mBERT, XLM-R). Or allow users to specify the language of documents and searches.

4.4.3 Scenario 3: Domain-Specific Vocabulary

A problem: Medical documents use domain terminology

The solution: Fine-tune embeddings on domain data

Cost: Higher token usage, more latency

Benefit: Better accuracy

Trade-off: Speed vs. accuracy

4.4.4 Scenario 4: Document Changes

A problem: Document updated, old embedding stale

The solution: Invalidate embedding on update

Overhead: Re-embedding cost

Timing: How soon to re-embed?

Strategy: Async re-embedding, gradual rollout

4.4.5 Scenario 5: Embedding Model Updates

A problem: We switch embedding models for better accuracy

The solution: Re-embed all documents

Overhead: Can take hours for large corpus

Complexity: Need to handle incremental updates

Strategy: Versioned embeddings, gradual migration

Chapter 5

Database Design Rationale

Let me walk you through why we designed the database the way we did.

5.1 Core Entities

5.1.1 Users

Every system needs users. The interesting thing is how we handle authentication and authorization.

```
CREATE TABLE users (  
  id UUID PRIMARY KEY,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  name VARCHAR(255) NOT NULL,  
  role VARCHAR(50) NOT NULL DEFAULT 'user',  
  is_active BOOLEAN DEFAULT true,  
  created_at TIMESTAMP,  
  updated_at TIMESTAMP,  
  deleted_at TIMESTAMP  
);
```

The `deleted_at` field deserves explanation. We use soft deletes. When a user deletes their account, we don't immediately delete all their data. We mark it as deleted and give them a 30-day grace period to change their mind. After 30 days, we permanently delete everything.

Why? Because:

1. It respects the GDPR right to be forgotten while giving users a recovery window
2. It makes accidents recoverable
3. It makes auditing easier
4. It gives you time to ensure all replicas are deleted

The `password_hash` field should never contain the actual password. We hash it with bcrypt. Even we don't know the user's password.

5.1.2 Buckets and Files

Files need to live in buckets (like folders in Google Drive).

```
CREATE TABLE buckets (  
  id UUID PRIMARY KEY,  
  user_id UUID NOT NULL,  
  name VARCHAR(255) NOT NULL,  
  is_public BOOLEAN DEFAULT false,  
  created_at TIMESTAMP
```

```
);  
  
CREATE TABLE files (  
  id UUID PRIMARY KEY,  
  bucket_id UUID NOT NULL,  
  name VARCHAR(255) NOT NULL,  
  size BIGINT NOT NULL,  
  mime_type VARCHAR(100),  
  drive_file_id VARCHAR(255),  
  checksum VARCHAR(128),  
  is_public BOOLEAN DEFAULT false,  
  created_at TIMESTAMP  
);
```

The `drive_file_id` is crucial. It's the ID of the file on Google Drive. When a user wants to download the file, we look up this ID and fetch the file from Google Drive.

The `checksum` is stored so we can verify file integrity. When a user downloads a file, we calculate the checksum and compare it with what we stored. If they don't match, we know the file got corrupted somehow.

The `is_public` flag is interesting. It allows users to share files with the world without authentication. A file can be publicly accessible while living in a private bucket. This is useful for things like sharing documents with non-users.

5.1.3 Dynamic Tables

Users can create custom tables with custom schemas.

```
CREATE TABLE user_tables (  
  id UUID PRIMARY KEY,  
  user_id UUID NOT NULL,  
  table_name VARCHAR(255) NOT NULL,  
  schema JSONB NOT NULL,  
  row_count INTEGER DEFAULT 0,  
  created_at TIMESTAMP  
);  
  
CREATE TABLE table_records (  
  id UUID PRIMARY KEY,  
  table_id UUID NOT NULL,  
  data JSONB NOT NULL,  
  created_at TIMESTAMP  
);
```

Here's where things get interesting. We're storing arbitrary JSON data. Users can define any schema they want, and we store the data as JSON. This is flexible but comes with trade-offs.

Benefit: Users can create any schema without us changing the application code. They can add fields without changing the database schema.

Trade-off: We lose some of the benefits of a relational database. We can't enforce types at the database level. We can't create indexes on fields within the JSON. Queries are slightly slower because we're doing JSON parsing in the query.

But for the MVP, this is a reasonable trade-off. We optimize for developer experience (users can define whatever schema they want) over raw performance.

5.1.4 Vector Collections

For semantic search.

```
CREATE TABLE vector_collections (  
  id UUID PRIMARY KEY,  
  user_id UUID NOT NULL,  
  name VARCHAR(255) NOT NULL,  
  dimension INTEGER NOT NULL,  
  distance_metric VARCHAR(50),  
  created_at TIMESTAMP  
);  
  
CREATE TABLE vector_documents (  
  id UUID PRIMARY KEY,  
  collection_id UUID NOT NULL,  
  doc_id VARCHAR(255),  
  content TEXT,  
  embedding_vector VECTOR(1536),  
  metadata JSONB,  
  created_at TIMESTAMP  
);
```

The `dimension` field stores the size of the embedding vector. OpenAI's Ada model produces 1536-dimensional vectors. If you use a different model with different dimensions, you specify that here.

The `embedding_vector` field is the actual vector. This requires the `pgvector` PostgreSQL extension.

The `metadata` stores arbitrary additional data about the document. It's searchable but not part of the vector. So you could store the document's source URL, creation date, author, etc.

5.2 Multi-Tenancy at the Database Level

Here's how we enforce tenant isolation at the database level:

```
CREATE POLICY files_tenant_isolation ON files  
FOR SELECT  
USING (  
  bucket_id IN (  
    SELECT id FROM buckets WHERE user_id = current_user_id()  
  )  
);
```

This policy says: when selecting files, only return files in buckets that belong to the current user. The database enforces this automatically. Even if a hacker tries to craft a query that bypasses the application logic, the database prevents them from seeing data they shouldn't.

Chapter 6

API Design: REST vs GraphQL

We're providing both REST and GraphQL APIs. This is intentional.

6.1 REST API: For Simple Cases

REST is straightforward. You want to upload a file? POST to `/api/v1/files/upload`. You want to list files? GET `/api/v1/files?bucket_id=123`.

REST is great because:

- It's simple
- Everyone understands it
- It works well for CRUD operations
- It's easy to test with curl

REST is bad because:

- Over-fetching (you get more data than you need)
- Under-fetching (you need to make multiple requests)
- Hard to handle complex queries
- Not great for mobile (where bandwidth is precious)

6.2 GraphQL API: For Complex Cases

GraphQL solves many of REST's problems.

Instead of multiple endpoints, you have one endpoint. Instead of getting all the data, you request only what you need.

```
query {  
  files(bucketId: "123", limit: 10) {  
    id  
    name  
    size  
    createdAt  
  }  
}
```

This query returns only the fields you ask for. No over-fetching.

GraphQL is great because:

- You get exactly what you ask for
- One endpoint for all queries

- Great for mobile (bandwidth efficient)
- Built-in introspection (API documentation)

GraphQL is bad because:

- More complex to implement
- Slightly slower than REST (parsing overhead)
- Can be abused (deeply nested queries)
- Requires more client-side sophistication

We provide both because different use cases benefit from different approaches.

6.3 Handling Complexity in Queries

One problem with GraphQL is that clients can ask for very complex queries. For example:

```
query {  
  files {  
    bucket {  
      user {  
        files {  
          bucket {  
            user {  
              ...  
            }  
          }  
        }  
      }  
    }  
  }  
}
```

This query is infinitely nested. The server would exhaust memory trying to fulfill it.

We handle this by:

1. Limiting query depth to 5 levels
2. Limiting the total number of nodes returned to 1000
3. Using query complexity scoring (some queries cost more than others)

Chapter 7

Handling Real-World Failures

Building for scale means building for failure. Things break. Networks fail. Services go down. Here's how we handle it.

7.1 Retry Logic

When you call Google Drive to upload a file and it fails, what do we do?

First attempt: Upload fails with error 500 (server error).

Our response: Wait 2 seconds, then try again.

If it fails again: Wait 4 seconds, then try again.

If it fails again: Wait 8 seconds, then try again.

We do this up to 5 times. This is exponential backoff. The idea is:

- If Google Drive is temporarily overloaded, waiting 2 seconds gives it time to recover
- If it's experiencing a larger outage, we don't hammer their servers
- We give the operation multiple chances to succeed
- But we don't wait forever

After 5 failed attempts, we give up and return an error to the user. We also log the error so we can investigate why Google Drive was unreliable.

7.2 Graceful Degradation

What if Google Drive API has a 99% success rate? That means 1% of uploads fail. For a system handling millions of uploads, that's a lot of failures.

We handle this by having replicas. When you upload a file, we upload it to multiple Google Drive accounts. If one fails, the others still succeed. The user gets a successful response because the primary copy was uploaded.

Later, in the background, we retry the failed replicas. Eventually, all replicas are created. This is automatic and the user doesn't see any of it.

7.3 Circuit Breakers

If Google Drive is having a major outage, we don't want to keep trying every request. That's wasteful.

So we implement a circuit breaker:

- When failure rate exceeds 50% for a specific contributor, we "trip the circuit"
- We stop sending requests to that contributor

- Other contributors handle the load
- Every 30 seconds, we test if the contributor is back online
- When it is, we resume using it

This protects Google’s servers from being hammered during an outage, and it protects our system from wasting time on failed requests.

Chapter 8

Multi-Tenancy in Practice

I want to talk more about multi-tenancy because it's crucial and it's where bugs hide.

8.1 The Tenant Context

Every request needs to know which tenant it belongs to. This is typically extracted from the JWT token. The JWT contains the user's ID and the tenant ID.

When a request comes in:

1. We validate the JWT signature
2. We extract the user ID and tenant ID
3. We set these in a ThreadLocal context
4. All subsequent code uses this context

Here's the critical part: we use a ThreadLocal, not a global variable. Why? Because the same thread might handle requests from different users over time. A ThreadLocal stores data per-thread, not globally.

```
// When request comes in
TenantContextHolder.setTenantId(tenantIdFromToken);

// In service code
UUID tenantId = TenantContextHolder.getTenantId(); // Gets the correct
    ↪ tenant

// After response is sent
TenantContextHolder.clear(); // Clean up
```

If we forgot that `clear()` call, the next request handled by this thread would have the wrong tenant context. Huge security bug.

8.2 Testing Tenant Isolation

How do you test that tenant isolation works? This is critical because it's a security feature.

Test Case 1: Tenant A tries to read Tenant B's data

- Create data in Tenant B
- Query as Tenant A
- Verify: Empty result set (no data)

Test Case 2: Tenant A tries to update Tenant B's data

- Create data in Tenant B
- Send update command as Tenant A
- Verify: Update fails (row not found)

Test Case 3: Tenant A tries to delete Tenant B's data

- Create data in Tenant B
- Send delete command as Tenant A
- Verify: Delete fails (row not found)

Test Case 4: SQL injection doesn't bypass isolation

- Query: "id=1 OR 1=1"
- Verify: Still filtered by tenant_id
- The database-level policy prevents escape

Test Case 5: Async jobs maintain tenant context

- Tenant A queues a job
- Verify: Job runs with Tenant A's context
- Tenant B's data isn't visible during job execution

Every single one of these needs to pass. If even one fails, you have a security hole.

Chapter 9

Scaling: How We Handle Growth

Let's talk about what happens as you grow from 10 users to 1,000 to 100,000 users.

9.1 Single Instance Phase (1-1,000 users)

Everything runs on one server. One Spring Boot instance. One PostgreSQL instance. One Redis instance.

At this scale, you don't need sophisticated scaling. You need simplicity. You want to be able to debug issues quickly. You want to understand exactly what's happening.

The bottleneck at this phase is usually: human bandwidth. Can you maintain it? Can you debug it? Can you add features?

9.2 Multi-Instance Phase (1,000-100,000 users)

Now you need multiple application instances. Users get routed to different instances via a load balancer.

The instances share the same PostgreSQL database. They share the same Redis instance.

The challenge here is: if you're not careful, you can create race conditions.

Example: Two users click "upload file" at exactly the same time. Both instances check available space in Redis. Both see that there's 100GB available. Both let the upload proceed. Now you've used 200GB but expected to use 100GB.

Solutions:

1. Use atomic operations in Redis (INCR, DECR) instead of read-then-write
2. Use database-level locks
3. Accept that you might overshoot quotas by a small amount and reconcile later

For MVP, option 3 is fine. You check quota, let the upload proceed, and if you went over, you make a note and plan to fix it later.

9.3 Distributed Phase (100,000+ users)

At this scale, you might want:

- Multiple PostgreSQL instances (with replication)
- Multiple Redis instances (with sharding)
- Multiple Weaviate instances
- Geographic distribution (users access servers near them)

This is complex. You need proper devops. You need monitoring. You need people who understand distributed systems.

We're not going to solve this in the MVP. But the architecture allows for it.

Chapter 10

Security: Layered Approach

Let's go deep on security. This is where you lose sleep at night if you get it wrong.

10.1 Authentication: Verifying Identity

A user claims to be Alice. How do we know they're really Alice?

10.1.1 Password-based auth

1. User submits email + password
2. We look up the email in the database
3. We check if the password hash matches
4. If yes, we're confident they're Alice

But passwords are weak. Users reuse them. They get phished. They're easy to guess.

10.1.2 Better: Multi-factor authentication

1. User submits email + password (something they know)
2. We send a code to their phone (something they have)
3. They enter the code
4. Now we're more confident they're Alice

But MFA is annoying for users. So we allow both: password + MFA is optional.

10.1.3 Tokens

Once we verify they're Alice, we issue them a token. This token proves they're Alice. They include this token in all subsequent requests.

The token is time-limited. After 15 minutes, it expires. They need to get a new one. This limits the damage if the token leaks.

10.2 Authorization: Verifying Permissions

A user proves they're Alice. But that doesn't mean Alice can do everything.

10.2.1 Resource ownership

Alice creates a file. She owns it. Only she can download, share, or delete it.

10.2.2 Permissions

Alice can share a file with Bob. She says “Bob can read this file but not delete it”. We store that permission. When Bob tries to download, we check: does Bob have read permission? Yes. Download allowed. When Bob tries to delete, we check: does Bob have delete permission? No. Delete denied.

10.2.3 Roles

Maybe Alice is an admin. We give her an “admin” role. This role has special permissions (like viewing usage statistics or resetting quotas).

Each of these needs to be checked before an action is allowed.

10.3 Data Protection: Encryption

Should we encrypt data at rest? This is a trade-off.

10.3.1 With encryption

- If someone steals the database, they see gibberish (good)
- But encryption/decryption takes CPU (slightly slower)
- And we need to manage encryption keys (complex)

10.3.2 Without encryption

- If someone steals the database, they see plaintext (bad)
- But no performance overhead
- Simpler to operate

For MVP, we don’t encrypt. For production, we recommend encryption for sensitive data. We could add it later.

For truly sensitive data (like API tokens), we encrypt them even in the MVP. We don’t want API tokens stored in plaintext.

10.4 Input Validation

Every input from the user is potentially malicious.

10.4.1 Example: File upload

- File size: Is it less than max size? If not, reject
- File type: Is it an allowed type? If not, reject
- File content: Does it contain malicious code? We scan it (future)

10.4.2 Example: Query parameter

- Limit: Is it a number between 1 and 100? If not, use default
- Offset: Is it a valid number? If not, use 0
- Filter: Is it a valid filter expression? If not, reject

For GraphQL queries, we also check:

- Query depth: Don't allow more than 5 levels of nesting
- Query complexity: Use a scoring system, reject complex queries
- Query size: Reject queries larger than 10KB

Chapter 11

Operational Reality: Running This Thing

Building CloudPool is one thing. Operating it is another.

11.1 Monitoring

You need to know when things go wrong. Ideally before users notice.

We track:

- **Request latency:** Are responses taking longer than usual?
- **Error rates:** What percentage of requests fail?
- **Database performance:** Are queries slow?
- **Queue depth:** How many jobs are waiting to be processed?
- **Storage usage:** How much of the pool is being used?
- **Google Drive quota:** Are we hitting rate limits?

When any of these metrics looks bad, we get an alert.

11.2 Debugging

When something goes wrong, you need to understand what happened.

We log:

- User ID and tenant ID with every request
- What operation was performed (upload, query, search)
- What the parameters were
- How long it took
- Whether it succeeded or failed
- If it failed, what was the error

With these logs, you can usually figure out what went wrong.

11.3 Maintenance Windows

PostgreSQL occasionally needs maintenance (vacuum, reindex). Google Drive API has occasional outages. You need to plan for this.

We:

- Schedule maintenance during low-traffic hours (late night)

- Communicate maintenance windows to users in advance
- Make sure backups are up to date before any risky operation
- Have a rollback plan if something goes wrong

Chapter 12

Edge Cases Nobody Expects

I could write a whole book about edge cases. Here are some that have surprised us:

12.1 Unicode in Filenames

User uploads a file with an emoji in the name: “[emoji]_photo.jpg”

This should work but doesn’t in some systems because:

- Some filesystems don’t support Unicode
- Some APIs don’t handle it properly
- String length calculations are wrong (emoji is multiple bytes)

Solution: Validate UTF-8, reject invalid Unicode, normalize Unicode (NFC vs NFD).

12.2 Concurrent Writes to Same Record

User A and User B both edit the same database record simultaneously. A makes one change, B makes another change. Whose change wins?

Option 1: Last write wins. B’s change overwrites A’s.

Option 2: Merge. If they edited different fields, both changes are kept.

Option 3: Conflict. Both fail. User has to resolve.

We go with option 1 for simplicity. But we could add optimistic locking later.

12.3 Accidental Sharing of Public Links

User creates a public link to share a file with a specific person. But the link is guessable. Another person guesses it and accesses the file.

Why is this a problem? Because the link might be to something sensitive.

Solution: Use non-guessable tokens. Generate a random 32-character string. That’s practically impossible to guess.

12.4 Files Disappearing

User uploads a file. Later, they download it. But the file isn’t there.

How does this happen?

1. File was deleted by mistake
2. Google Drive contributor left the pool and took their data
3. File corrupted and replicas failed to restore

4. Bug in our code

We handle this by:

- Soft deletes (30-day recovery window)
- Replication (multiple copies)
- Checksum validation
- Alerting on anomalies

But sometimes files legitimately disappear. We need to be honest about this and handle it gracefully.

Chapter 13

The Path From MVP to Production

You can't build everything at once. So here's the realistic path:

13.1 Phase 1: MVP

- Users can sign up
- Users can upload files (to their own storage only, no pooling yet)
- Users can share files with others
- PostgreSQL works
- Basic monitoring

Not in Phase 1:

- Google Drive pooling (complex)
- Semantic search (complex)
- Admin dashboard (not critical)
- Advanced analytics (not critical)

13.2 Phase 2: Make it Real

- Google Drive pooling (fully working)
- Replication across contributors
- Error handling and retries
- Better monitoring

13.3 Phase 3: Add Features

- Semantic search
- Custom tables (for relational data)
- REST API (we started with basic REST)
- Pagination and filtering

13.4 Phase 4: Scale

- Database optimization
- Caching layer
- Multi-instance deployment

- Load testing

13.5 Phase 5+: Harden

- Security audit
- Compliance (GDPR, etc)
- Better documentation
- Admin tools
- Community outreach

This is realistic. We Build incrementally. Get feedback. Iterate.

Chapter 14

Cost Analysis: Is This Actually Free?

Let me be honest about the costs.

14.1 Storage

Free (Google Drive free tier)

14.2 Compute

You need a server to run CloudPool. Options:

- Self-hosted: You have a server, so \$0 for us
- Cloud server: \$20-50/month for basic plan
- Managed service: \$100-500/month depending on features

14.3 Database

PostgreSQL is free if you run it yourself. Or \$15-50/month for a managed instance.

14.4 Embeddings

If you use OpenAI for semantic search, \$0.02 per 1M tokens (cheap but not free)

14.5 Bandwidth

If you're self-hosted, depends on your ISP. If you're on cloud, depends on provider.

14.6 Real cost for self-hosted

\$100-200/month (server + bandwidth)

14.7 Real cost for managed service

\$500-1000/month (simpler to operate)

14.8 AWS equivalent cost

\$5000-10000/month

14.9 Savings

10-50x cheaper than cloud

This is still a massive saving. But it's not literally free.